╰(..:: The Ultimate JavaScript Notebook [PART 4] ::..)╯

~ By Sagar Biswas

Browser Object Model (BOM) - Location Object

The **Location Object** is part of the Browser Object Model (BOM) and provides information about the current URL of the document. It also allows you to manipulate the URL (ex: **redirecting to a new page**).

Location Object Properties

The **location** object (or **window.location**) contains several useful properties:

Property	Description
location.href	Returns the full URL of the current page.
location.hostname	Returns the domain name (e.g., "www.example.com").
location.pathname	Returns the path of the URL (e.g., "/index.html").
location.protocol	Returns the protocol (e.g., "http:" or "https:").
location.port	Returns the port number (if specified, e.g., "8080").
location.search	Returns the query string (e.g., "?id=123").
location.hash	Returns the anchor part of the URL (e.g., "#section1").

Location Object Methods

Method	Description
location.assign(url)	Loads a new document (**redirects to the given URL**).
location.reload()	Reloads the current page.
location.replace(url)	Replaces the current document with a new one (**does not store in history**).

Example: Using the Location Object

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Location Object Example</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 800px;
            margin: 0 auto;
            /* margin used to center the content */
            padding: 20px;
        }

        button {
            padding: 10px 15px;
            margin: 5px;
            cursor: pointer;
            /* cursor: pointer is used to change the cursor to a pointer when hovering over the button */

        }

        #warning {
            color: red;
            font-weight: bold;
        }
    </style>
```

Sagar Biswas

```html
</head>

<body>
    <div id="container">
        <h1>Browser Object Model (BOM) - Location Object</h1>
        <p>The <strong>Location Object</strong> provides information about the current URL and allows navigation.</p>

        <div id="url-info">
          <!-- span works like a div but it is inline. means it will not create a new line and strong will make the
text bold -->
            <p><strong>Current URL:</strong> <span id="href"></span></p>
            <p><strong>Hostname:</strong> <span id="hostname"></span></p>
            <p><strong>Pathname:</strong> <span id="pathname"></span></p>
            <p><strong>Protocol:</strong> <span id="protocol"></span></p>
            <p><strong>Port:</strong> <span id="port"></span></p>
            <p><strong>Query String:</strong> <span id="search"></span></p>
        </div>

        <div>
            <button id="assignBtn">Visit YouTube (assign)</button>
            <button id="replaceBtn">Visit Google (replace)</button>
            <button id="reloadBtn">Reload Page</button>
        </div>

        <p id="warning">Warning: The "replace" button will navigate to Google WITHOUT adding to browser history!</p>
    </div>

    <script>
        // Log the window and location objects
        console.log("Window object:", window);   // window is a global object
        console.log("Location object:", location);       // same as window.location

        // Display location properties
        document.getElementById("href").textContent = location.href;
        document.getElementById("hostname").textContent = location.hostname;
        document.getElementById("pathname").textContent = location.pathname;
        document.getElementById("protocol").textContent = location.protocol;
        document.getElementById("port").textContent = location.port || "Not specified"; // if the location.port is not
specified, it will show "Not specified"
        document.getElementById("search").textContent = location.search || "None"; // if the location.search is not
specified, it will show "None"

        // Redirect to YouTube using assign() (adds to history)
        document.getElementById("assignBtn").addEventListener("click", function () {
            console.log("Using assign() - will add to browser history");
            location.assign("https://www.youtube.com");
        });

        // Redirect to Google using replace() (does NOT add to history)
        document.getElementById("replaceBtn").addEventListener("click", function () {
            console.log("Using replace() - will NOT add to browser history");
            location.replace("https://www.google.com");
            // after visiting google, we can't go back to the previous page.
        });

        // Reload the page
        document.getElementById("reloadBtn").addEventListener("click", function () {
            location.reload();
        });
    </script>
</body>

</html>
```

Output:

Browser Object Model (BOM) - Location Object

The **Location Object** provides information about the current URL and allows navigation.

Current URL: http://127.0.0.1:3000/Lession%201%20--%20First%20JavaScript%20Program/index.html

Hostname: 127.0.0.1

Pathname: /Lession%201%20--%20First%20JavaScript%20Program/index.html

Protocol: http:

Port: 3000

Query String: None

Visit YouTube (assign) | Visit Google (replace) | Reload Page

Warning: The "replace" button will navigate to Google WITHOUT adding to browser history!

Sagar Biswas

- **Key Differences Between** assign() **and** replace()

Feature	**location.assign(url)**	**location.replace(url)**
History	Adds a new entry in the browser history.	Replaces the current entry (no back button).
Use Case	Normal page navigation.	**Preventing** users from going back (ex: after logout).

- **Practical Use Cases**

1. **Page Redirection**
   - Use location.assign() to navigate to a new page. (ex: redirecting after login.
2. **Force Page Reload**
   - Use location.reload() to refresh the page. (e.g., after form submission).
3. **Security**
   - Use location.replace() to prevent users from going back (ex: after logout).

Browser Object Model (BOM) - Popup Boxes

Popup boxes are used to interact with users through simple dialog windows. JavaScript provides three main types of popup boxes: alert(), confirm(), and prompt().

- **Types of Popup Boxes**

Method	**Description**	**Returns**
alert(message)	Displays a message with an OK button.	**undefined**
confirm(message)	Displays a message with OK and Cancel buttons.	**true** (OK) or **false** (Cancel)
prompt(message, default)	Displays a message with a text input field and OK/Cancel buttons.	**string** (input value) or **null** (Cancel)

Example: Using Popup Boxes

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Popup Boxes Example</title>
</head>
<body>
    <h1>Popup Boxes in JavaScript</h1>
    <p>Check the console for outputs.</p>

    <script>
        // 1. alert() - Shows a message with an OK button
        alert("Warning: This action cannot be undone!");

        // 2. confirm() - Asks for user confirmation (OK/Cancel)
        function deleteItem() {
            let shouldDelete = confirm("Are you sure you want to delete this item?");
            if (shouldDelete) { // if OK Clicked (ture)
                console.log("Item deleted.");
            } else { // Cancel
                console.log("Deletion cancelled.");
            }
        }
        deleteItem();

        // 3. prompt() - Asks for user input
        function welcomeUser() {
```

Sagar Biswas

```
            let userName = prompt("Enter your name:", "Guest");
            let greeting = userName ? `Welcome, ${userName}!` : "Welcome, Guest!";

            let heading = document.createElement("h2");
            heading.textContent = greeting;
            document.body.appendChild(heading);
        }
        welcomeUser();
    </script>
</body>
</html>
```

Practical Use Cases

1. alert()
   - Show warnings (ex: "**Invalid input!**").
   - Display success messages (ex: "**Data saved successfully!**").
2. confirm()
   - Confirm critical actions (ex: "**Are you sure you want to delete this file?**").
   - Handle user decisions (ex: "**Proceed to checkout?**").
3. prompt()
   - Get user input (ex: "**Enter your age:**"). Takes input as **string**.
   - Set **default** values (ex: **prompt("Enter name:", "Guest")**).

Key Takeaways

- alert() → Simple notifications.
- confirm() → Yes/No decisions.
- prompt() → User input collection.
- Popup boxes **block JavaScript execution** until dismissed.
- **Avoid overusing popups (they can annoy users).**

JavaScript Notebook: Browser Object Model (BOM) - Timing Events

Timing events in JavaScript allow you to execute code after a specified time interval or repeatedly at fixed intervals. The **two main methods** for timing events are:

1. **setTimeout()** - Executes a function once after a delay.
2. **setInterval()** - Repeatedly executes a function at a given interval.

1. setTimeout()

Executes a function **once** after a specified delay (in milliseconds).

Syntax

```
setTimeout(function, delayInMilliseconds, [arg1, arg2, ...]);
```

Example:

```
setTimeout(() => {
    console.log("Hello after 2 seconds!");
}, 2000);
```

Clearing a Timeout

Sagar Biswas

Use clearTimeout() to cancel a timeout before it executes.

☠ Example:

```
const timeoutId = setTimeout(() => {
    console.log("This will not run");
}, 2000);

clearTimeout(timeoutId); // it stops the timeoutId's 2s timeout
        console.log("Timeout cleared");
```

2. setInterval()

Executes a function **repeatedly** at a specified interval (in milliseconds).

⇨ **Syntax**

```
setInterval(function, intervalInMilliseconds, [arg1, arg2, ...]);
```

🔥 **Practical Example** (setTimeout and setInterval())

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Timing Events Example</title>
</head>
<body>
    <button class="save-btn">Save (setTimeout)</button>
    <button class="save-btn1">Start Counter (setInterval)</button>
    <p class="message"></p>

    <script>
        const btn = document.querySelector('.save-btn');
        const btn1 = document.querySelector('.save-btn1');
        const message = document.querySelector('.message');

        // setTimeout Example
        btn.addEventListener('click', () => {
            message.textContent = "Data saved successfully!";
            setTimeout(() => {
                message.textContent = "";                        // Clear message after 2 seconds
            }, 2000);
        });

        // setInterval Example
        btn1.addEventListener('click', () => {
            let count = 0;
            message.textContent = count;

            const intervalId = setInterval(() => {
                count++;
                message.textContent = count;

                if (count >= 5) {                                // Stop after 5 counts
                    clearInterval(intervalId);             // Use clearInterval() to stop the interval.
                    message.textContent += " (Stopped)";  // += to append text to the existing message
                }
            }, 1000);                                            // Update every 1 second
        });
    </script>
</body>
</html>
```

🔥 **Key Differences**

Feature	**setTimeout**()	**setInterval()**
Execution	Runs **once** after a delay.	Runs **repeatedly** at fixed intervals.
Stopping	Use **clearTimeout(timeoutId).**	Use **clearInterval(intervalId).**
Use Case	Delayed actions (ex: **notifications**).	**Animations, real-time updates, timers.**

Jagri Biswas

Practical Use Cases

1. setTimeout()
    - Show a temporary success message.
    - Delay an action (ex: redirect after 5 seconds).
2. setInterval()
    - Create a countdown timer.
    - Update live data (ex: stock prices).
    - Simple animations.

Key Takeaways

- setTimeout() → Runs code **once** after a delay.
- setInterval() → Runs code **repeatedly** at intervals.
- Always **clear intervals/timeouts** when no longer needed to avoid memory leaks.
- Useful for animations, notifications, and real-time updates.

Browser Object Model | Digital Clock Implementation

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Digital Clock</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }

        button {
            padding: 10px 20px;
            font-size: 16px;
            cursor: pointer;
            background-color: #4CAF50;
            color: white;
            border: none;
            border-radius: 5px;
        }

        #time-display {
            font-size: 24px;
            margin: 20px 0;
        }

        #formatted-time {
            font-size: 20px;
            color: #333;
        }
    </style>
</head>

<body>
    <button id="start-clock">Start Digital Clock</button>
    <div id="time-display"></div>
    <div id="formatted-time"></div>

    <script>
        // DOM Elements
        const startButton = document.getElementById('start-clock');
        const timeDisplay = document.getElementById('time-display');
        const formattedDisplay = document.getElementById('formatted-time');
```

Sagar Biswas

```
        // Clock state
        let clockInterval; // used to store the interval ID
        let isRunning = false; // used to check if the clock is running or not

        // Event Listener for Start Button
        startButton.addEventListener('click', function () {
            if (!isRunning) { // Check if the clock is already running
                startClock(); // calling function
                isRunning = true;
                startButton.textContent = 'Clock Running...';
                startButton.disabled = true; // used to disable the Clock Running... button
            }
        });

        // Function to format time values (add leading zero)
        function formatTime(value) {
            return value < 10 ? `0${value}` : value; // if value is less than 10, add a leading zero

        }

        // Function to start the clock
        function startClock() {

            // Initial update
            updateClock(); // calling function

            clockInterval = setInterval(updateClock, 1000); // Update the time every second
        }

        // Function to get current time and update displays
        function updateClock() {

            const now = new Date();

            // Full date and time string
            timeDisplay.textContent = `Current Date & Time: ${now.toString()}`;

            // Formatted date and time
            const day = formatTime(now.getDate()); // Get current date-time
            const month = formatTime(now.getMonth() + 1); // getMonth returns 0-11, so we add 1 to get 1-12
            const year = now.getFullYear();
            const minutes = formatTime(now.getMinutes());
            const seconds = formatTime(now.getSeconds());

            // let hours = formatTime(now.getHours()); // causes a logic bug when calculating AM/PM and adjusting hours
            let hours = now.getHours(); //  <-- keeping raw value first. Getting current hours (0-23)

            // Determine AM/PM
            const period = hours >= 12 ? 'PM' : 'AM';

            // Convert to 12-hour format
            hours = hours % 12;
            hours = hours ? hours : 12; // 0 becomes 12
            hours = formatTime(hours); // <-- format after conversion

            formattedDisplay.innerHTML = `
                Formatted Date: ${day}/${month}/${year}. And
                Formatted Time: ${hours}:${minutes}:${seconds} ${period}
                `;
        }

        // Bonus: Stop the clock when leaving the page
        window.addEventListener('beforeunload', function () {
            if (clockInterval) { // used to check if the clock is running or not
                clearInterval(clockInterval);
            }
        });
    </script>
</body>

</html>
```

Output:

Clock Running...

Current Date & Time: Tue Apr 29 2025 13:34:45 GMT+0600 (Bangladesh Standard Time)

Formatted Date: 29/04/2025. And Formatted Time: 01:34:45 PM

---------- X ----------

Sagar Biswas